🧩 1. What is a Class Variable?

A **class variable** is a variable that is **shared by all objects** of a class.

It belongs to the **class itself**, not to any particular instance (object).

Think of it like:

"A class variable is a common property for *all* objects."

🧱 2. What is an Instance Variable?

An **instance variable** belongs to a specific object (instance).
Each object has its **own copy** of that variable.

💡 Quick Comparison

Type	Defined	Belongs To	Shared	Example
Class Variable	Inside class, outside methods	Class	✅ Shared	Car.wheels
Instance Variable	Inside __init__()	Object	❌ Not Shared	self.color

---

🚙 3. Example: Car Class

Let's create a Car class to understand both types 👇

```
class Car:
    wheels = 4  # Class variable (shared)

    def __init__(self, brand, color):
        self.brand = brand  # Instance variable
        self.color = color  # Instance variable

# Create two car objects
```

```
car1 = Car("Toyota", "Red")
car2 = Car("Tesla", "Black")

print("Car1:", car1.brand, car1.color, "Wheels:", car1.wheels)
print("Car2:", car2.brand, car2.color, "Wheels:", car2.wheels)
```

📜 **Output:**

```
Car1: Toyota Red Wheels: 4
Car2: Tesla Black Wheels: 4
```

✅ Both cars share the same **class variable** wheels.

🔧 4. Changing Variables

Now, let's modify them and observe what happens 👇

```
car1.wheels = 6       # modifies only car1's copy
Car.wheels = 5        # modifies class variable for all objects

print(car1.wheels)    # 6 (instance-specific)
print(car2.wheels)    # 5 (from class)
print(Car.wheels)     # 5
```

🧠 **Key Point:**
When you assign car1.wheels = 6, Python **creates a new instance variable** for car1.
It does **not** modify the class variable.

🧠 5. How Python Looks for a Variable (Resolution Order)

When you access car1.wheels:

1. Python first checks if car1 (the object) has wheels defined.
2. If not, it checks the class Car.

3. If still not found, it looks in parent classes (inheritance).

🧰 6. Example: Counting Objects (Real Use Case)

You can use class variables to **track common data** for all instances.

```
class Student:
    count = 0  # Class variable to count objects

    def __init__(self, name):
        self.name = name
        Student.count += 1  # increment every time a new student is created

s1 = Student("Alice")
s2 = Student("Bob")
s3 = Student("Charlie")

print("Total Students:", Student.count)
```

📜 **Output:**

```
Total Students: 3
```

✅ Each time you create a new object, the class variable updates for *all* objects.

🧱 7. Class Variable with Class Methods

You can access and modify class variables using a **class method**.

```
class Employee:
    company_name = "TechCorp"

    @classmethod
```

```
    def change_company(cls, new_name):
        cls.company_name = new_name

e1 = Employee()
e2 = Employee()

print(Employee.company_name)  # TechCorp
Employee.change_company("NextGenSoft")
print(e1.company_name)       # NextGenSoft
print(e2.company_name)       # NextGenSoft
```

🧠 *cls refers to the class itself — just like self refers to the object.*

🧪 8. Practice Exercise

Try this on your own 👇

```
# Exercise: Python Class Variable Practice
# 1. Create a class 'Library'
#    - class variable: total_books = 0
# 2. Every time a new book is added (through __init__), increment total_books
# 3. Create 3 books and print total_books
# 4. Add a class method 'update_books' to modify total_books manually
```

📜 9. Summary Table

Type	Declared In	Accessed By	Shared	Example
Instance Variable	__init__()	self.variable	❌ No	self.color
Class Variable	Class block	Class.variable or cls.variable	✅ Yes	Car.wheels

---

🎓 **Instructor Tip**

Think of it like this:

🏠 Class variables = “Apartment Building Features” (shared by all flats)
🏡 Instance variables = “Flat-specific details” (different for each)